

Research Ideas & Exploration Plan

QUIC Server-Side Migration Project

Prepared for Professor | June 19, 2026

Current Status

We have a **fully working cross-machine QUIC server migration** implementation using modified Mozilla Neco. The system migrates live QUIC connections from a primary server to a preferred server using the RFC 9000 Preferred Address mechanism. Verified with Firefox over HTTP/3. Five state transfer backends have been implemented and tested (TCP Push, HTTP Pull, Redis KV, Redis Pub/Sub, File). Fatima has completed the Docker-based implementation with network namespace isolation.

Component	Status	Owner
Cross-machine migration (Neco/Rust)	COMPLETE	Anik
5 state transfer backends	COMPLETE	Anik
Firefox HTTP/3 verification	COMPLETE	Anik
Docker / namespace setup	COMPLETE	Fatima
/tmp folder migration	IN PROGRESS	Fatima
TCP state transfer (Docker)	IN PROGRESS	Fatima

Research Task 1: State Transfer Backend Comparison Study

Goal: Systematic comparison of /tmp (shared filesystem), TCP, QUIC, and Redis as state transfer mechanisms for migration state, evaluating security, performance, scalability, and detectability.

Motivation

Different state transfer backends create fundamentally different threat models. A /tmp-based transfer leaves forensic artifacts on disk. TCP creates observable side-channel connections. QUIC-over-QUIC nests the state inside encrypted traffic. Redis introduces a third-party dependency. Understanding these tradeoffs is critical for both attackers (stealth) and defenders (detection).

Backends to Compare

/tmp (Shared Filesystem)

- State written to shared mount between containers or namespaces
- Zero network footprint for transfer itself

- Leaves disk forensic artifacts (inode timestamps, file contents)
- Only viable when servers share a filesystem (same host, NFS, shared volume)

TCP Push

- Primary opens TCP connection to preferred server, pushes state
- Observable side-channel: firewall/IDS can see TCP connection between servers
- Lowest latency (~1ms LAN), simplest implementation
- Already implemented and tested in our Neqo codebase

QUIC as State Transfer Channel

- Use QUIC itself to transfer migration state between servers
- State transfer is encrypted and indistinguishable from regular QUIC traffic
- Hardest for IDS to detect (encrypted, looks like normal traffic)
- Added complexity: servers need their own QUIC connection

Redis (KV and Pub/Sub)

- Centralized state store decouples primary and preferred servers
- Scalable to many servers, supports lazy retrieval
- Introduces third-party infrastructure dependency
- Network observer sees Redis protocol traffic, not migration state directly

Research Questions

- RQ1:** Which backend is most stealthy from a network forensics perspective?
- RQ2:** What is the latency overhead of each backend, and does it affect migration success rate?
- RQ3:** Can a network observer distinguish migration state transfer from legitimate traffic?
- RQ4:** Which backend is most resilient to partial failures (e.g., preferred server not ready)?
- RQ5:** How does QUIC-over-QUIC compare to TCP in terms of detectability?

Methodology

Run each backend under controlled conditions. Capture traffic with tcpdump. Measure: (1) transfer latency, (2) migration success rate, (3) network fingerprint visibility, (4) disk/memory forensic artifacts. Use Wireshark dissectors to analyze whether an observer can identify the state transfer.

Research Task 2: Pass-Through Handshake with Secondary Server

Goal: Explore a modified QUIC handshake where the client communicates with the primary server, but the handshake simultaneously involves the secondary (preferred) server, so the secondary is cryptographically ready from the start.

Motivation

In our current implementation, the primary server completes the full TLS handshake, then exports ~445 bytes of crypto state to the preferred server. This introduces a window where the preferred server has no crypto context. A **pass-through handshake** model would let the secondary server participate in the handshake from the beginning, eliminating the state transfer delay and reducing the window of vulnerability.

Approach

The client sends its Initial packet to the primary server. The primary forwards the handshake messages to the secondary in real-time. The secondary derives its own crypto keys from the shared handshake. When migration occurs, the secondary already has the keys -- no post-handshake state export needed.

Key Design Options

Option A: Primary as Proxy

- Primary relays all handshake packets to secondary in parallel
- Secondary silently derives keys but does not send packets
- On migration, secondary is immediately ready
- Advantage: Client is completely unaware, zero protocol changes
- Challenge: Primary must forward packets with minimal latency

Option B: Shared TLS Session

- Primary and secondary share the same TLS master secret from the start
- Both servers can independently derive traffic keys
- Requires pre-shared key material or coordinated key generation
- Advantage: True zero-delay migration
- Challenge: Sharing master secrets weakens forward secrecy guarantees

Option C: Delegated Credentials (RFC 9345)

- Primary issues a delegated credential to the secondary
- Secondary can authenticate as the primary to the client
- Standard-compliant approach
- Advantage: No custom crypto, works with existing TLS stacks
- Challenge: Delegated credentials authenticate identity, not session state

Research Questions

RQ1: Can we achieve zero-delay migration by involving the secondary in the handshake?

RQ2: What are the security implications of sharing handshake state in real-time?

RQ3: Does pass-through handshake make the attack more or less detectable?

RQ4: How does this compare to our current post-handshake state export approach?

Research Task 3: Multi-Hop Chained Migration

Goal: Investigate whether a QUIC connection can be migrated through a chain of servers ($A \rightarrow B \rightarrow C \rightarrow \dots$), creating a multi-hop exfiltration path analogous to Tor-style onion routing.

Motivation

Our current implementation migrates from one server to another (single hop). But the QUIC Preferred Address mechanism does not inherently prevent the preferred server from issuing its own preferred address, creating a chain. This raises the question: can an attacker bounce a connection across multiple servers to obscure the final destination?

Research Questions

- RQ1:** Does RFC 9000 allow or prevent chained preferred address migrations?
- RQ2:** How many hops can a connection survive before the client times out or gives up?
- RQ3:** Does each hop increase detection difficulty, or does the pattern become more visible?
- RQ4:** Can chained migration be used for geographic evasion (cross jurisdictional boundaries)?

Approach

Modify the preferred server to itself advertise a new preferred address upon receiving the migrated connection. Each server in the chain exports state to the next. Test with 3-4 servers on the LAN, measure cumulative latency and client behavior.

Research Task 4: Network Forensics & Indistinguishability Analysis

Goal: Rigorously analyze whether a malicious server migration (QUIC-Exfil) is distinguishable from a legitimate preferred address migration at the network layer.

Motivation

The QUIC-Exfil paper showed ML classifiers achieve only 0-47% F1-score at detecting malicious migrations. We now have real packet captures from physical hardware. This task goes deeper: analyze exactly what a network observer can and cannot see, and formally characterize the indistinguishability properties.

Analysis Dimensions

- Packet-level analysis:** Compare packet sizes, timing, and ordering between legitimate and malicious migrations. Are there statistical differences?
- Side-channel visibility:** Does the state transfer (TCP/Redis/QUIC) between servers create a detectable side-channel? Characterize per backend.

TLS fingerprinting: Can the preferred server's TLS behavior be distinguished from the primary's? (e.g., different cipher negotiation, certificate handling)

Timing correlation: Can an observer correlate the timing of the state transfer with the PATH_CHALLENGE/RESPONSE exchange?

Deliverables

Packet capture analysis report. Feature comparison table (legitimate vs malicious). Statistical tests on timing distributions. Recommendations for what defenders should monitor (if anything can help).

Research Task 5: Defense Mechanisms & Protocol-Level Mitigations

Goal: Propose and evaluate concrete defenses against QUIC-Exfil, ranging from RFC amendments to middlebox strategies to client-side protections.

Motivation

Demonstrating the attack is necessary but not sufficient for a strong paper. Proposing defenses shows the research is constructive, not just offensive. This positions the work as responsible disclosure with actionable recommendations.

Proposed Defenses to Evaluate

Preferred Address Subnet Pinning

- Restrict preferred_address to the same subnet or AS as the primary
- Client or middlebox rejects migration to a different network
- Breaks legitimate CDN use cases but prevents cross-network exfiltration

Certificate Binding

- Require the preferred server to present the same certificate as the primary
- Prevents migration to a server with a different identity
- Already partially implied by QUIC spec, but not enforced in practice

Client-Side Migration Policy

- Browser setting to disable or restrict preferred_address migrations
- User/admin can whitelist trusted migration targets
- Analogous to HSTS pinning for migration destinations

Middlebox PATH_CHALLENGE Interception

- Firewall inspects PATH_CHALLENGE/RESPONSE for anomalous patterns
- Challenge: QUIC encrypts these frames, so deep inspection is limited
- Could use timing heuristics instead of payload inspection

RFC Amendment: Migration Attestation

- Propose a new transport parameter: migration_attestation
- Primary server provides a signed proof that preferred server is authorized
- Client can verify the attestation before migrating
- Requires PKI infrastructure but provides strong guarantees

Research Task 6: Cross-Network (WAN) Migration Feasibility

Goal: Test whether QUIC server migration works across different subnets, WANs, or even across the public Internet, fundamentally changing the threat model.

Motivation

Our current testbed is on a single LAN (141.217.168.x). The QUIC-Exfil threat model becomes significantly more dangerous if migration works across the Internet, because data could be exfiltrated to a server in a different country or jurisdiction. Testing WAN migration also reveals NAT traversal and firewall challenges.

Experiments

- Same campus, different subnets (different VLANs)
- Cross-campus or cross-institution (VPN or public IP)
- Cloud-to-cloud (e.g., AWS us-east to AWS eu-west)
- Measure: added latency, NAT/firewall interference, PATH_CHALLENGE timeout rates

Research Task 7: Exfiltration Capacity & Covert Channel Analysis

Goal: Quantify the practical bandwidth of the QUIC-Exfil covert channel. How much data can be exfiltrated per migration, and can repeated migrations create a sustained covert channel?

Key Questions

- Migration state is ~445 bytes. Can additional data be embedded in unused fields?
- Can the preferred server serve different content than the primary intended?
- Can repeated short-lived connections each trigger a migration, creating throughput?
- What is the theoretical maximum bandwidth of this covert channel?
- How does this compare to other known covert channels (DNS tunneling, ICMP, etc.)?

Summary: All Research Tasks at a Glance

#	Task	Type	Priority	Effort
1	State Transfer Comparison (tmp, TCP, QUIC, Redis)	Engineering + Analysis	HIGH	Medium
2	Pass-Through Handshake with Secondary	Protocol Design	HIGH	High
3	Multi-Hop Chained Migration	Exploration	MEDIUM	Medium
4	Network Forensics & Indistinguishability	Security Analysis	HIGH	Medium
5	Defense Mechanisms & Mitigations	Defense	HIGH	Medium
6	Cross-Network (WAN) Migration	Feasibility Study	MEDIUM	Low-Med
7	Exfiltration Capacity & Covert Channel	Threat Analysis	MEDIUM	Low

Recommended Focus for Paper

For the strongest research contribution, we recommend combining **Tasks 1, 2, 4, and 5** into a cohesive paper narrative:

- **Attack implementation** (already done) -- prove QUIC-Exfil is feasible on real hardware
- **Task 1** -- show attackers have multiple backend choices with different stealth properties
- **Task 2** -- demonstrate an advanced variant (pass-through handshake) that eliminates state transfer delay
- **Task 4** -- prove the attack is indistinguishable from legitimate migration
- **Task 5** -- propose concrete, actionable defenses (responsible disclosure angle)

Tasks 3, 6, and 7 are valuable explorations that can strengthen the paper as additional results or serve as future work directions.